

## Problema : N + 1 query

Sarcina A - Demonstratie problema:

```
...  
e1_0.department_id=?
```

## Sarcina B: Rezolvați cu Eager Loading

```
long start = System.currentTimeMillis();

try (Session session = sf.openSession()) {
    List<Department> departments = session
        .createQuery(
            queryString: "SELECT DISTINCT d FROM Department d LEFT JOIN FETCH d.employees",
            Department.class)
        .getResultList();

    System.out.printf("Loaded %d Department rows WITH their employees (1 query)",
        departments.size());

    for (Department dept : departments) {
        List<Employee> employees = dept.getEmployees();
        System.out.printf("    %-28s - %d employees%n",
            dept.getName(), employees.size());
    }
}

long elapsed = System.currentTimeMillis() - start;
```

[Hibernate]

```
select
    distinct d1_0.id,
    e1_0.department_id,
    e1_0.id,
    e1_0.email,
    e1_0.name,
    e1_0.salary,
    d1_0.name
from
    departments d1_0
left join
    employees e1_0
    on d1_0.id=e1_0.department_id
```

```
Prepared SQL statements : 1
JPQL queries executed   : 1
Entities loaded         : 10020
Execution time          : 155 ms
```

## 2. Analiza Performanței Indexării

Sarcina A: Benchmark Fără Index-uri

Metodele timeOne si timeTwo executa query-urile sql Q1, Q2, Q3, Q4 si returneaza timpul mediu de executie pentru fiecare. TimeOne = query cu un parametru si timeTwo = query cu doi parametri

```
private static double timeOne(SessionFactory sf, String sql, String p1, Object v1) { 2 usages
    long total = 0;
    try (StatelessSession ss = sf.openStatelessSession()) {
        for (int i = 0; i < ITERATIONS; i++) {
            long t0 = System.nanoTime();
            ss.createNativeQuery(sql).setParameter(p1, v1).getResultList();
            total += System.nanoTime() - t0;
        }
    }
    return total / (double) ITERATIONS / 1_000_000.0;
}

private static double timeTwo(SessionFactory sf, String sql, 2 usages
    String p1, Object v1, String p2, Object v2) {
    long total = 0;
    try (StatelessSession ss = sf.openStatelessSession()) {
        for (int i = 0; i < ITERATIONS; i++) {
            long t0 = System.nanoTime();
            ss.createNativeQuery(sql).setParameter(p1, v1).setParameter(p2, v2).getResultList();
            total += System.nanoTime() - t0;
        }
    }
    return total / (double) ITERATIONS / 1_000_000.0;
}
```

De asemenea planul de executie eafisat folosing explain analyze

```
private static void runExplainAnalyze(SessionFactory sf, Long deptId, String email) { 2 usages
    String[] queryLabels = {
        "Q1 - email search",
        "Q2 - department_id search",
        "Q3 - salary BETWEEN 50000 AND 80000",
        "Q4 - department_id + salary > 60000 (multi-column)"
    };
    String[] explainSql = {
        "EXPLAIN ANALYZE SELECT * FROM employees WHERE email = '" + email + "'",
        "EXPLAIN ANALYZE SELECT * FROM employees WHERE department_id = " + deptId,
        "EXPLAIN ANALYZE SELECT * FROM employees WHERE salary BETWEEN 50000 AND 80000",
        "EXPLAIN ANALYZE SELECT * FROM employees WHERE department_id = " + deptId + " AND salary > 60000"
    };

    try (Session session = sf.openSession()) {
        for (int i = 0; i < explainSql.length; i++) {
            System.out.println("\n>> " + queryLabels[i]);
            /unchecked/
            List<String> plan = session.createNativeQuery(explainSql[i], String.class)
                .getResultList();
            plan.stream().limit(maxSize: 6).forEach(line -> System.out.println("    " + line));
        }
    }
}
```

Cand nu avem index putem observa ca se face seq scan adica se merge prin toata tabela

Execution Time: 0.754 ms

## Sarcina B: Adăugați Index-uri Strategice

```
[DDL] CREATE INDEX IF NOT EXISTS idx_employees_dept_salary ON employees(department_id, salary)
```

DUpa adaugare de indexuri strategice si rulara din nou a benchmark-ului observam ca se face index scan

loops=1)

## Tabel Rezultate Așteptate:

| Interogare       | Fara Index | Cu Index | Imbunatatire |
|------------------|------------|----------|--------------|
| Cautare email    | 2.016      | 1.503    | 1.3          |
| Cautare dep      | 3.241      | 1.969    | 1.6          |
| Interval salariu | 6.291      | 1.527    | 4.1          |
| Multi col        | 2.516      | 1.601    | 1.5          |

### 3. Implementare Paginare

```
public class Page<T> { 6 usages

    private final List<T> content; 3 usages
    private final int    pageSize; 5 usages
    private final int    pageNumber; // -1 when keyset 4 usages
    private final long   totalElements; // -1 when keyset 5 usages
    private final Long   lastId; // null when offset 3 usages
    private final boolean hasNext; 3 usages

    public Page(List<T> content, int pageNumber, int pageSize, long totalElements) { 1 usage
        this.content      = content;
        this.pageNumber    = pageNumber;
        this.pageSize      = pageSize;
        this.totalElements = totalElements;
        this.lastId        = null;
        this.hasNext       = (long)(pageNumber + 1) * pageSize < totalElements;
    }

    public Page(List<T> content, int pageSize, Long lastId, boolean hasNext) { 1 usage
        this.content      = content;
        this.pageNumber    = -1;
        this.pageSize      = pageSize;
        this.totalElements = -1;
        this.lastId        = lastId;
        this.hasNext       = hasNext;
    }

    public List<T> getContent() { return content; } 2 usages
    public int     getPageNumber() { return pageNumber; } no usages
    public int     getPageSize() { return pageSize; } no usages
    public long    getTotalElements() { return totalElements; } 1 usage
    public long    getTotalPages() { return totalElements < 0 ? -1 : (totalElements + pageSize - 1) / pageSize; } 1 usage
    public Long    getLastId() { return lastId; } 1 usage
    public boolean hasNext() { return hasNext; } no usages
    public boolean hasPrevious() { return pageNumber > 0; } no usages
}
```

Clasa page retine informatii specifice in functie de tipul de paginare daca e cu offset atunci se retine pageNumber , totalElements si lastId e null , iar daca e cu keyset atunci se retine lastId si pageNumber si totalElements sunt -1

Strategia A: Paginare Bazată pe Offset (LIMIT/OFFSET)S

```

public Page<Employee> getPage(int pageNumber, int pageSize) { 9 usages
    try (Session session = sf.openSession()) {
        List<Employee> employees = session
            .createQuery( queryString: "SELECT e FROM Employee e ORDER BY e.id", Employee.class)
            .setFirstResult(pageNumber * pageSize)
            .setMaxResults(pageSize)
            .getResultList();

        long total = session
            .createQuery( queryString: "SELECT COUNT(e) FROM Employee e", Long.class)
            .getSingleResult();

        return new Page<>(employees, pageNumber, pageSize, total);
    }
}

```

Strategia B: Paginare Bazată pe Cursor (Keyset)

```

public Page<Employee> getPageAfter(Long lastId, int pageSize) { 9 usages
    try (Session session = sf.openSession()) {
        // Fetch one extra row to detect whether a next page exists.
        List<Employee> raw = session
            .createQuery(
                queryString: "SELECT e FROM Employee e WHERE e.id > :lastId ORDER BY e.id",
                Employee.class)
            .setParameter( parameter: "lastId", lastId)
            .setMaxResults(pageSize + 1)
            .getResultList();

        boolean hasNext = raw.size() > pageSize;
        List<Employee> employees = hasNext ? new ArrayList<>(raw.subList(0, pageSize)) : raw;
        Long newLastId = employees.isEmpty() ? lastId : employees.get(employees.size() - 1).getId();

        return new Page<>(employees, pageSize, newLastId, hasNext);
    }
}

```

Am implementat ambele strategii in UI

Paginare Employees - Strategii Compare

Configurare

☒ Strategie A - Offset (LIMIT/OFFSET)
 ☐ Strategie B - Keyset (Cursor/Seek)
 Dimensiune pagina: 25 Total: 10,000 inregistrari

| #  | ID | Nume                | Email       | Dept ID | Salariu |
|----|----|---------------------|-------------|---------|---------|
| 1  | 1  | user_1@example.com  | Employee_1  | 6       | 47,224  |
| 2  | 2  | user_2@example.com  | Employee_2  | 4       | 117,193 |
| 3  | 3  | user_3@example.com  | Employee_3  | 8       | 110,200 |
| 4  | 4  | user_4@example.com  | Employee_4  | 3       | 81,381  |
| 5  | 5  | user_5@example.com  | Employee_5  | 9       | 85,670  |
| 6  | 6  | user_6@example.com  | Employee_6  | 1       | 90,962  |
| 7  | 7  | user_7@example.com  | Employee_7  | 7       | 107,192 |
| 8  | 8  | user_8@example.com  | Employee_8  | 18      | 46,065  |
| 9  | 9  | user_9@example.com  | Employee_9  | 19      | 81,583  |
| 10 | 10 | user_10@example.com | Employee_10 | 19      | 86,714  |
| 11 | 11 | user_11@example.com | Employee_11 | 10      | 59,275  |
| 12 | 12 | user_12@example.com | Employee_12 | 1       | 109,827 |
| 13 | 13 | user_13@example.com | Employee_13 | 7       | 103,292 |
| 14 | 14 | user_14@example.com | Employee_14 | 1       | 49,483  |
| 15 | 15 | user_15@example.com | Employee_15 | 5       | 95,800  |
| 16 | 16 | user_16@example.com | Employee_16 | 8       | 100,183 |
| 17 | 17 | user_17@example.com | Employee_17 | 1       | 43,301  |
| 18 | 18 | user_18@example.com | Employee_18 | 14      | 66,027  |
| 19 | 19 | user_19@example.com | Employee_19 | 20      | 66,406  |

< Anterior Pagina 1 din 400 Urmator >

Benchmark:

```
return new Double[]{first, mid, last},  
}
```

Resultate:

Exaplin analyze:

Execution Time: 0.029 ms

Comaparatie:

Paginarea cu offset e constant pentru primele pagini si  $O(n)$  in adancime. La keyset e logn pentru a foloseste index seek si constatn indiferent de adancime. Folosim cu offset cand avem nevoie de numarul total de inregistrari si trebuie sa putem sari de la o pagina la alta. Keyset pentru infinite scroll

#### 4. Implementare Caching

```
@Entity
@Table(name = "departments")
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Department {
```

Cache miss: prima încărcare din baza de date

```
private static void taskA(SessionFactory sf, Statistics stats) { 1 usage
    stats.clear();
    System.out.println("\n Loading departments 1-5 (cache is empty):\n");

    for (long id = 1; id <= 5; id++) {
        long hitsBefore = stats.getSecondLevelCacheHitCount();
        long missesBefore = stats.getSecondLevelCacheMissCount();

        long t0 = System.nanoTime();
        try (Session s = sf.openSession()) {
            Department d = s.get(Department.class, id);
            double ms = (System.nanoTime() - t0) / 1e6;

            boolean hit = stats.getSecondLevelCacheHitCount() > hitsBefore;
            boolean miss = stats.getSecondLevelCacheMissCount() > missesBefore;
            System.out.printf(" dept #%-2d '%-18s' - %-4s (%.3f ms)%n",
                id, d.getName(), miss ? "MISS" : hit ? "HIT" : "???", ms);
        }
    }
    printRegionStats(sf);
}
```

```
dept #1 'Department_1_updated' - MISS (87.400 ms)
dept #2 'Department_2' - MISS (3.595 ms)
dept #3 'Department_3' - MISS (3.043 ms)
dept #4 'Department_4' - MISS (2.926 ms)
dept #5 'Department_5' - MISS (2.720 ms)
```

Cache hit: încărcările ulterioare din cache

```
Re-loading the same departments 1-5 (should all be HIT):

dept #1 'Department_1_updated_updated' - HIT (2.028 ms)
dept #2 'Department_2' - HIT (0.382 ms)
dept #3 'Department_3' - HIT (0.289 ms)
dept #4 'Department_4' - HIT (0.299 ms)
dept #5 'Department_5' - HIT (0.278 ms)
```

```
Cache region stats [org.example.Department]
Hits: 5 | Misses: 0 | Puts: 0 | Hit Rate: 100.0%
```

```
private static void taskB(SessionFactory sf, Statistics stats) { 1 usage
    stats.clear();
    System.out.println("\n Re-loading the same departments 1-5 (should all be HIT):\n");

    for (long id = 1; id <= 5; id++) {
        long hitsBefore = stats.getSecondLevelCacheHitCount();
        long missesBefore = stats.getSecondLevelCacheMissCount();

        long t0 = System.nanoTime();
        try (Session s = sf.openSession()) {
            Department d = s.get(Department.class, id);
            double ms = (System.nanoTime() - t0) / 1e6;

            boolean hit = stats.getSecondLevelCacheHitCount() > hitsBefore;
            boolean miss = stats.getSecondLevelCacheMissCount() > missesBefore;
            System.out.printf(" dept #%-2d '%-18s' - %-4s (%.3f ms)%n",
                id, d.getName(), hit ? "HIT " : miss ? "MISS" : "???", ms);
        }
    }
    printRegionStats(sf);
}
```

Evicțiunea cache-ului: actualizarea declanșează reîmprospătarea cache-ului

```
dept #1 renamed to 'Department_1_updated_updated_updated'  
In L2 cache after commit: true (READ_WRITE refreshed it)
```

Step 2: Reload dept #1 - should be HIT with new name.

```
dept #1 = 'Department_1_updated_updated_updated' - HIT (0.384 ms)
```

Step 3: Explicit eviction via `sf.getCache().evict(Department.class, 1L)`.

In L2 cache after evict: false

```
dept #1 re-load - MISS (3.559 ms) - fetched from DB
```

Cache region stats [org.example.Department]

```
Hits: 2 | Misses: 1 | Puts: 2 | Hit Rate: 66.7%
```

```

// 1. Update dept #1 inside a transaction (READ_WRITE: cache updated on commit)
System.out.println("\n Step 1: Update dept #1 inside a transaction.");
String updatedName;
try (Session s = sf.openSession()) {
    s.beginTransaction();
    Department d = s.get(Department.class, id: 1L);
    updatedName = d.getName() + "_updated";
    d.setName(updatedName);
    s.getTransaction().commit();
}
System.out.printf(" dept #1 renamed to '%s'\n", updatedName);
System.out.printf(" In L2 cache after commit: %b (READ_WRITE refreshed it)\n",
    sf.getCache().contains(Department.class, primaryKey: 1L));

// 2. Reload - should be HIT with updated value
System.out.println("\n Step 2: Reload dept #1 - should be HIT with new name.");
long hitsBefore = stats.getSecondLevelCacheHitCount();
try (Session s = sf.openSession()) {
    long t0 = System.nanoTime();
    Department d = s.get(Department.class, id: 1L);
    double ms = (System.nanoTime() - t0) / 1e6;
    boolean hit = stats.getSecondLevelCacheHitCount() > hitsBefore;
    System.out.printf(" dept #1 = '%s' - %-4s (%.3f ms)\n",
        d.getName(), hit ? "HIT " : "MISS", ms);
}

// 3. Explicit eviction - next load must hit the database
System.out.println("\n Step 3: Explicit eviction via sf.getCache().evict(Department.class);");
sf.getCache().evict(Department.class, primaryKey: 1L);
System.out.printf(" In L2 cache after evict: %b\n",
    sf.getCache().contains(Department.class, primaryKey: 1L));

long missesBefore = stats.getSecondLevelCacheMissCount();
try (Session s = sf.openSession()) {
    long t0 = System.nanoTime();
    Department d = s.get(Department.class, id: 1L);
    double ms = (System.nanoTime() - t0) / 1e6;
    boolean miss = stats.getSecondLevelCacheMissCount() > missesBefore;
    System.out.printf(" dept #1 re-load - %-4s (%.3f ms) - fetched from DB\n",
        miss ? "MISS" : "HIT?", ms);
}
printRegionStats(sf);

```

Expirarea TTL: invalidarea automată a cache-ului

```
// — Task D: wait for TTL, then show automatic eviction —————

private static void taskD(SessionFactory sf, Statistics stats) { 1 usage
    stats.clear();

    // Pre-fill cache with dept #2
    try (Session s = sf.openSession()) {
        s.get(Department.class, id: 2L);
    }
    System.out.printf("%n dept #2 loaded into cache. TTL = %ds.%n", TTL_SECS);
    System.out.printf("  In cache: %b%n", sf.getCache().contains(Department.class, primaryKey: 2L));
    System.out.printf("  Sleeping %d seconds ...%n", TTL_SECS + 1);

    try { Thread.sleep(millis: (TTL_SECS + 1) * 1000L); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }

    System.out.printf("  In cache after TTL: %b (auto-evicted)%n",
        sf.getCache().contains(Department.class, primaryKey: 2L));

    long missesBefore = stats.getSecondLevelCacheMissCount();
    try (Session s = sf.openSession()) {
        long t0 = System.nanoTime();
        Department d = s.get(Department.class, id: 2L);
        double ms = (System.nanoTime() - t0) / 1e6;
        boolean miss = stats.getSecondLevelCacheMissCount() > missesBefore;
        System.out.printf("  dept #2 reload after TTL - %-4s (%.3f ms) - from DB%n",
            miss ? "MISS" : "HIT?", ms);
    }
    printRegionStats(sf);
}
}
```

## 5. Optimizarea Operațiilor în Masă

### Abordarea 1: Actualizări Individuale

```
private static void approach1(SessionFactory sf) { 1 usage
    try (Session s = sf.openSession()) {
        s.beginTransaction();
        List<Employee> emps = s
            .createQuery(queryString: "SELECT e FROM Employee e", Employee.class)
            .getResultList();
        for (Employee emp : emps) {
            emp.setSalary(emp.getSalary() * RAISE);
        }
        s.getTransaction().commit();
    }
}
}
```

```
BULK UPDATE BENCHMARK - Salary raise 10%  
Rows: 10,000 | Batch size: 50 | Iterations: 3
```

Approach 1 - Individual updates (dirty checking, N UPDATES)

iter 1 : 683 ms

iter 2 : 625 ms

iter 3 : 557 ms

Average : 622.1 ms

Abordarea 2: Interogare Actualizare în Masă

```
private static void approach2(SessionFactory sf) { 1 usage  
    try (Session s = sf.openSession()) {  
        s.beginTransaction();  
        s.createMutationQuery( hqlString: "UPDATE Employee e SET e.salary = e.salary * :r")  
            .setParameter( name: "r", RAISE)  
            .executeUpdate();  
        s.getTransaction().commit();  
    }  
}
```

iter 1 : 229 ms

iter 2 : 226 ms

iter 3 : 234 ms

Average : 229.9 ms

Abordarea 3: Actualizări Batch

iter 1 : 49 ms

iter 2 : 36 ms

iter 3 : 39 ms

Average : 41.8 ms

```

private static void approach3(SessionFactory sf) { 1 usage
    try (Session s = sf.openSession()) {
        s.beginTransaction();

        int offset = 0;
        List<Employee> batch;
        do {
            batch = s.createQuery( queryString: "SELECT e FROM Employee e ORDER BY e.id", Employee.class)
                .setFirstResult(offset)
                .setMaxResults(BATCH_SIZE)
                .getResultList();

            for (Employee emp : batch) {
                emp.setSalary(emp.getSalary() * RAISE);
            }

            s.flush();
            s.clear();
            offset += batch.size();
        } while (batch.size() == BATCH_SIZE);

        s.getTransaction().commit();
    }
}

```

RESULTS (3 iterations, 10,000 rows, +10% raise)

| Approach                        | Avg (ms) | vs Bulk  |
|---------------------------------|----------|----------|
| 1 - Individual (dirty checking) | 540.0 ms | baseline |
| 2 - Bulk JPQL UPDATE            | 200.5 ms | 2.7x     |
| 3 - Batch flush/clear/50        | 41.8 ms  | 12.9x    |

**Abordarea Individuală** este cea mai lentă deoarece încarcă, modifică și salvează fiecare entitate separat, actualizând și memoria cache. Este recomandată doar pentru modificări foarte mici (sub 100 de înregistrări).

**Abordarea Bulk JPQL** este cea mai rapidă metodă. Ea trimite o comandă de Update direct către baza de date, dar ignoră complet memoria cache și regulile de business (evenimentele) din aplicația Java. Este ideală când vrei doar să schimbi date rapid.

**Abordarea Batch (în calupuri)** reprezintă calea de mijloc perfectă pentru volume mari de date. Prin procesarea pe bucăți (ex: câte 50 de entități), protejează memoria RAM a aplicației de supraîncărcare, dar permite în același timp executarea logicii și a validărilor pentru fiecare obiect în parte.

## 6. Caching-ul Prepared Statements

Test A: Fără Reutilizare (crearea de statement-uri noi)

```
private static double taskA(SessionFactory sf) { 1 usage
    long total = 0;
    for (int iter = 0; iter < ITERS; iter++) {
        long t0 = System.nanoTime();
        try (Session s = sf.openSession()) {
            for (int i = 0; i < QUERIES; i++) {
                s.createQuery(HQL, Employee.class)
                    .setParameter(parameter: "id", (long) i)
                    .getSingleResultOrNull();
            }
        }
        long elapsed = System.nanoTime() - t0;
        total += elapsed;
        System.out.printf("  iter %d : %,d ms%n", iter + 1, elapsed / 1_000_000);
    }
    double avg = total / (double) ITERS / 1_000_000.0;
    System.out.printf("  Medie  : %.1f ms%n", avg);
    return avg;
}
```

Test B: Cu Reutilizare Statement

```

private static double taskB(SessionFactory sf) { 1 usage
    long total = 0;
    for (int iter = 0; iter < ITERS; iter++) {
        long t0 = System.nanoTime();
        try (Session s = sf.openSession()) {
            var query = s.createQuery(HQL, Employee.class);
            for (int i = 0; i < QUERIES; i++) {
                query.setParameter("id", (long) i);
                query.getSingleResultOrNull();
            }
        }
        long elapsed = System.nanoTime() - t0;
        total += elapsed;
        System.out.printf(" iter %d : %d ms%n", iter + 1, elapsed / 1_000_000);
    }
    double avg = total / (double) ITERS / 1_000_000.0;
    System.out.printf(" Medie : %.1f ms%n", avg);
    return avg;
}

```

Task A - fara reutilizarea statement-urilor

```

iter 1 : 2,209 ms
iter 2 : 1,405 ms
iter 3 : 1,268 ms
iter 4 : 1,359 ms
iter 5 : 1,231 ms
Medie : 1494.9 ms

```

Task B - cu reutilizarea statement-urilor

```

iter 1 : 1,253 ms
iter 2 : 1,192 ms
iter 3 : 1,299 ms
iter 4 : 1,262 ms
iter 5 : 1,228 ms
Medie : 1247.4 ms

```

COMPARATIE (1000 query-uri x 5 iteratii)

| Varianta                                      | Medie (ms) |
|-----------------------------------------------|------------|
| A - fara reutilizare (createQuery() per call) | 1494.9 ms  |
| B - cu reutilizare (Query object refolosit)   | 1247.4 ms  |

A: la fiecare apel createQuery() Hibernate compileaza HQL -> SQL si cauta planul in query-plan-cache (overhead la fiecare query).

B: compilarea are loc o singura data; setParameter() inlocuieste doar valoarea parametrului, planul SQL este reutilizat

Implementați caching rezultate interogări la nivel ORM (+5)

```
hibernate.cache.use_query_cache=true
```

Step 1: cacheable JPQL query - first call (MISS).

Returned 5 rows in 301.982 ms - MISS

Step 2: identical query - second call (HIT, no DB round-trip).

Returned 5 rows in 2.517 ms - HIT

Query cache hits : 1

Query cache misses: 1

Query cache puts : 1

Step 3: any write to 'departments' auto-invalidates query cache.

UPDATE committed - query cache region timestamp advanced.

Step 4: repeat query after invalidation (MISS again).

Returned 5 rows in 6.974 ms - MISS (cache invalidated)

TTL expiration EhCache auto-evicts after 30s

Global L2 hits : 0

Global L2 misses: 0

Global L2 puts : 5

Query cache hits : 1

Query cache misses: 2

Adăugați monitoring/metrici pentru operații bază de date (+5)